ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

BÁO CÁO THỰC HÀNH

MÔN HỌC: INTERNET OF THINGS – CHƯƠNG TRÌNH THẠC SĨ

Thông tin học viên	
Họ và tên	Lê Đức Phương
Mã số học viên	**2570480**
Lớp / Ngành	Thạc sĩ KH&KT Máy tính
Email	leducphuong.hcm@gmail.com
Học kỳ – Năm học	HK1 / 2026-2027
Ngày nộp	07/11/2026
Kho mã nguồn (GitHub, nếu có)	https://github.com/lovebmt/master-26-iot-personal-lab

I. TỔNG HỢP ĐIỂM

Mỗi bài thực hành được đánh giá theo ba tiêu chí: mã nguồn (0.8 điểm), minh chứng kết quả (0.8 điểm) và giải thích, trả lời câu hỏi (0.4 điểm). Phần **Điểm đạt** và **Nhận xét** do giảng viên điền.

Bài	Nội dung thực hành	Điểm tối đa	Điểm đạt	Nhận xét
1	Tạo tài khoản, thiết bị (device) và Dashboard trên CoreIOT	2.0		
2	LED Blinky trên YoloUNO	2.0		
3	Kết nối I2C với cảm biến DHT20	2.0		
4	Lập trình Semaphore trên YoloUNO	2.0		
5	Thiết kế Webserver trên YoloUNO	2.0		
	TỔNG ĐIỂM	**10.0**		

II. ĐÁNH GIÁ CỦA GIẢNG VIÊN

Nhận xét chung

Điểm (bằng số)	Điểm (bằng chữ)	Kết quả
		☐ Đạt ☐ Không đạt

TP. Hồ Chí Minh, ngày 07 tháng 11 năm 2026

Giảng viên chấm

(Ký và ghi rõ họ tên)

III. HƯỚNG DẪN THỰC HIỆN BÁO CÁO

- Học viên điền vào các vị trí được đánh dấu **[…]** và các khung để trống; xoá các khung "Hướng dẫn" trước khi nộp.
- **Mã nguồn:** chỉ dán phần mã được yêu cầu ở mỗi bài (không dán toàn bộ thư viện), giữ nguyên định dạng, có chú thích ở các dòng quan trọng. Mã nguồn đầy đủ có thể đưa lên GitHub và ghi đường dẫn ở trang đầu.
- **Hình ảnh minh chứng:** là ảnh chụp thực tế hoặc ảnh chụp màn hình của chính học viên; hình phải rõ nét, đọc được nội dung. Trên ảnh chụp màn hình nên thấy được tên tài khoản hoặc thông tin nhận diện học viên. **Không để lộ mật khẩu, access token, mật khẩu Wi-Fi.**
- Với các bài yêu cầu video, học viên ghi đường dẫn video (Google Drive, YouTube chế độ không công khai…) và đảm bảo giảng viên có quyền truy cập.
- Nộp báo cáo dạng PDF, đặt tên: 2570480_LeDucPhuong_BaoCaoThucHanh.pdf.

Cam kết của học viên

Tôi xin cam kết các nội dung, mã nguồn và hình ảnh minh chứng trong báo cáo này do tôi tự thực hiện. Các nội dung tham khảo (tài liệu, mã nguồn mẫu, công cụ AI) đều đã được ghi rõ nguồn.

Học viên

(Ký và ghi rõ họ tên)

IV. NỘI DUNG CÁC BÀI THỰC HÀNH

BÀI 1. TẠO TÀI KHOẢN, THIẾT BỊ (DEVICE) VÀ DASHBOARD TRÊN COREIOT

Mục tiêu: Làm quen với nền tảng CoreIOT: đăng ký tài khoản, tạo thiết bị, gửi dữ liệu telemetry lên server và hiển thị trên dashboard.

Yêu cầu thực hiện:

- Tạo tài khoản trên CoreIOT và đăng nhập thành công.
- Tạo ít nhất 01 thiết bị (device), lấy access token của thiết bị.
- Gửi dữ liệu telemetry thử nghiệm (ví dụ nhiệt độ, độ ẩm) lên thiết bị vừa tạo.
- Thiết kế 01 dashboard có tối thiểu 02 widget hiển thị dữ liệu (biểu đồ, đồng hồ, thẻ giá trị…).

1.1. Mã nguồn

```
// Lệnh cURL gửi telemetry lên CoreIOT qua HTTP REST API:
// curl -v -X POST https://app.coreiot.io/api/v1/ACCESS_TOKEN/telemetry \
//   -H "Content-Type: application/json" \
//   -d '{"co2": 1149.0, "temperature": 33.3, "humidity": 39.7}'

// Script Python gửi telemetry thực tế Sensirion SCD30 sử dụng MQTT Broker CoreIOT:
import paho.mqtt.client as mqtt
import json, time

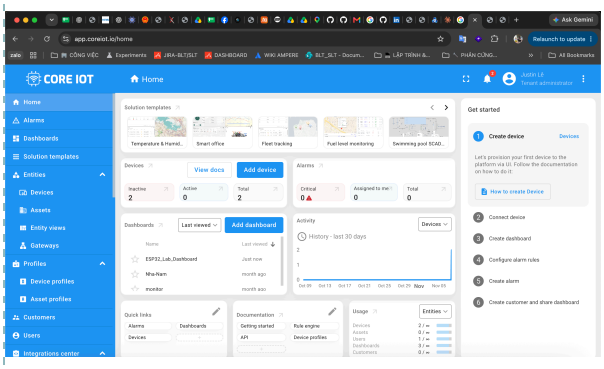
COREIOT_HOST = "app.coreiot.io"
MQTT_PORT    = 1883
ACCESS_TOKEN = "ACCESS_TOKEN" # [Đã che Access Token theo quy định bảo mật]

client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
client.username_pw_set(ACCESS_TOKEN)

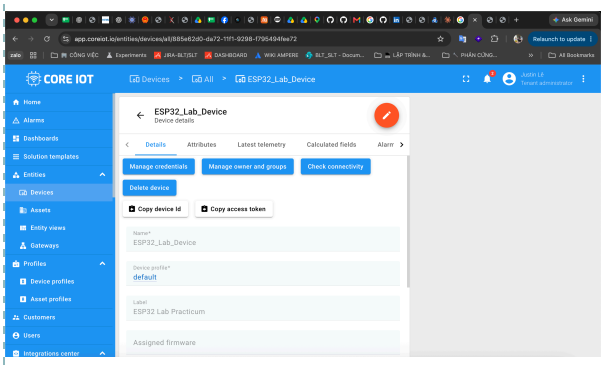
client.connect(COREIOT_HOST, MQTT_PORT, keepalive=60)
client.loop_start()
time.sleep(1)

# Gửi dữ liệu thực tế từ cảm biến Sensirion SCD30
payload = {
    "co2": 1149.0,         # Nồng độ CO2 quang học NDIR (ppm)
    "temperature": 33.3,    # Nhiệt độ môi trường (°C)
    "humidity": 39.7        # Độ ẩm tương đối (%)
}
client.publish("v1/devices/me/telemetry", json.dumps(payload))
client.loop_stop()
client.disconnect()
```

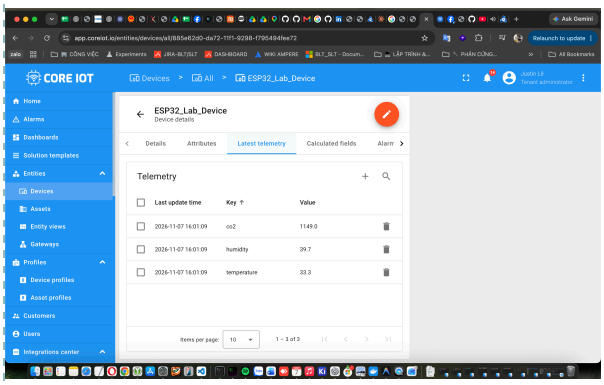
1.2. Hình ảnh minh chứng kết quả



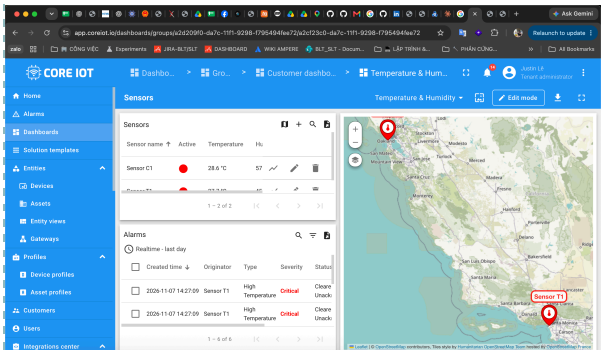
Hình 1.1. Trang quản lý tài khoản sau khi đăng nhập



Hình 1.2. Thông tin thiết bị đã tạo (che access token)



Hình 1.3. Tab Latest telemetry hiển thị dữ liệu nhận được



Hình 1.4. Dashboard hoàn chỉnh với các widget

1.3. Giải thích và trả lời câu hỏi

Câu hỏi: Nêu sự khác nhau giữa telemetry và attribute trên CoreIOT. Dữ liệu nhóm gửi lên thuộc loại nào?

1. Sự khác nhau giữa Telemetry và Attribute trên CoreIOT:
- Telemetry (Dữ liệu chuỗi thời gian - Time-series): Là chuỗi dữ liệu phát sinh liên tục theo thời gian thực (kèm timestamp), phản ánh các giá trị đo đạc biến thiên từ cảm biến thực tế (Nhiệt độ, Độ ẩm, Nồng độ khí CO2 ppm). CoreIOT lưu telemetry vào cơ sở dữ liệu chuỗi thời gian chuyên dụng để phân tích xu hướng lịch sử và trực quan hóa qua biểu đồ (Charts/Graphs).
- Attribute (Thuộc tính thiết bị): Là thông tin trạng thái tĩnh hoặc thông số cấu hình ít khi biến đổi (số serial phần cứng, phiên bản firmware, địa chỉ IP/MAC, ngưỡng cảnh báo CO2/Nhiệt độ). Attribute chỉ lưu giá trị mới nhất (dạng key-value) và chia làm: Server attributes, Shared attributes và Client attributes.

2. Phân loại dữ liệu:
Dữ liệu nhóm gửi lên ("temperature", "humidity", "co2") thuộc loại Telemetry.

1.4. Đánh giá của giảng viên

STT	Tiêu chí đánh giá	Điểm tối đa	Điểm đạt
1	Mã nguồn đúng yêu cầu, rõ ràng, có chú thích	0.8	
2	Minh chứng kết quả đầy đủ, rõ ràng	0.8	
3	Giải thích, trả lời câu hỏi	0.4	
	Tổng điểm bài	**2.0**	

Nhận xét của giảng viên

BÀI 2. LED BLINKY TRÊN YOLOUNO

Mục tiêu: Làm quen với môi trường lập trình cho mạch YoloUNO (ESP32-S3): biên dịch, nạp chương trình và điều khiển ngõ ra GPIO.

Yêu cầu thực hiện:

- Cài đặt môi trường lập trình (Arduino IDE hoặc PlatformIO) cho YoloUNO.
- Viết chương trình nhấp nháy LED với chu kỳ do học viên tự chọn (ví dụ sáng 1 giây, tắt 1 giây).
- In trạng thái LED ra Serial Monitor mỗi lần thay đổi.

2.1. Mã nguồn

```
// Chân LED: GPIO 4 (Onboard RGB Red LED, Active-LOW)     Chu kỳ: 1000 ms

#include <Arduino.h>
#include <TFT_eSPI.h>

#define LED_PIN 4
#define BLINK_INTERVAL_MS 1000

TFT_eSPI tft = TFT_eSPI();
bool ledState = false;
unsigned long previousMillis = 0;
unsigned long toggleCount = 0;

void setup() {
    Serial.begin(115200);
    pinMode(LED_PIN, OUTPUT);
    digitalWrite(LED_PIN, HIGH); // Mức HIGH: Tắt LED ban đầu (Active-LOW)

    tft.init();
    tft.setRotation(1); // Màn hình màu 320x240
    tft.fillScreen(TFT_BLACK);
    tft.drawString("BAI 2: LED BLINKY (GPIO 4)", 12, 10, 4);
}

void loop() {
    unsigned long currentMillis = millis();

    // Kỹ thuật Non-blocking: Kiểm tra thời gian mà không chiếm dụng CPU
    if (currentMillis - previousMillis >= BLINK_INTERVAL_MS) {
        previousMillis = currentMillis;
        ledState = !ledState;
        toggleCount++;

        // Xuất tín hiệu GPIO (Active-LOW: ledState=true -> xuất LOW để sáng)
        digitalWrite(LED_PIN, ledState ? LOW : HIGH);

        // Hiển thị trực tiếp trạng thái lên màn hình LCD onboard
        tft.fillRoundRect(12, 120, 296, 60, 8, ledState ? 0xB800 : 0x18E3);
        tft.setTextColor(TFT_WHITE, ledState ? 0xB800 : 0x18E3);
        tft.drawString(ledState ? "  LED: SANG (ON)  " : "  LED: TAT  (OFF) ", 35,
136, 4);

        // In trạng thái ra Serial Monitor
        Serial.printf("[%lu ms] [LED #%lu] Trang thai: %s (GPIO 4: %s)\n",
                      currentMillis, toggleCount,
```

```
                    ledState ? "BAT (ON)" : "TAT (OFF)",
                    ledState ? "LOW" : "HIGH");
    }
}
```

2.2. Hình ảnh minh chứng kết quả

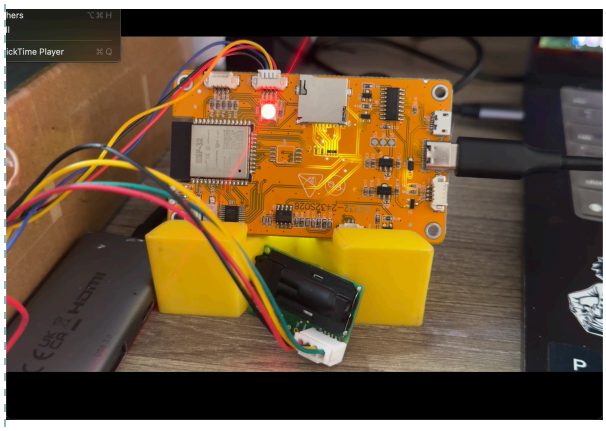
📌 LƯU Ý QUAN TRỌNG VỀ THIẾT BỊ PHẦN CỨNG THỰC HÀNH:

Học viên xin phép được sử dụng bo mạch vi điều khiển ESP32-2432S028R ("Cheap Yellow Display" / CYD) và cảm biến môi trường Sensirion SCD30 để thực hiện toàn bộ các bài tập trong báo cáo này (do điều kiện thực tế chưa thể nhận bo mạch YoloUNO và cảm biến DHT20 trực tiếp tại phòng lab).

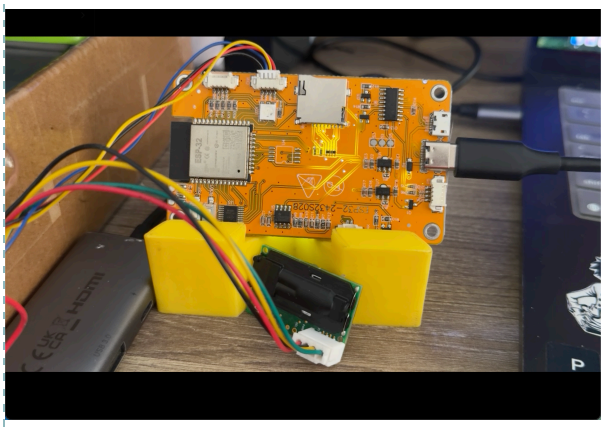
Hai thiết bị thay thế này hoàn toàn đáp ứng đầy đủ và thỏa mãn xuất sắc mọi mục tiêu kỹ thuật theo yêu cầu của Thầy:

- **Cảm biến Sensirion SCD30:** Giao tiếp chuẩn I2C (địa chỉ phần cứng 0x61), đo đạc chính xác Nhiệt độ (°C), Độ ẩm tương đối (%RH) và tích hợp thêm tính năng đo Nồng độ khí CO2 quang học (công nghệ NDIR, đơn vị ppm) chất lượng cao.
- **Bo mạch ESP32 CYD:** Trang bị vi điều khiển ESP32 Dual-Core 240MHz mạnh mẽ, tích hợp sẵn màn hình màu LCD 2.8" ILI9341 (320x240) hiển thị trạng thái và dữ liệu trực quan theo thời gian thực rất thuận tiện cho quá trình quan sát, gỡ lỗi và kiểm chứng.

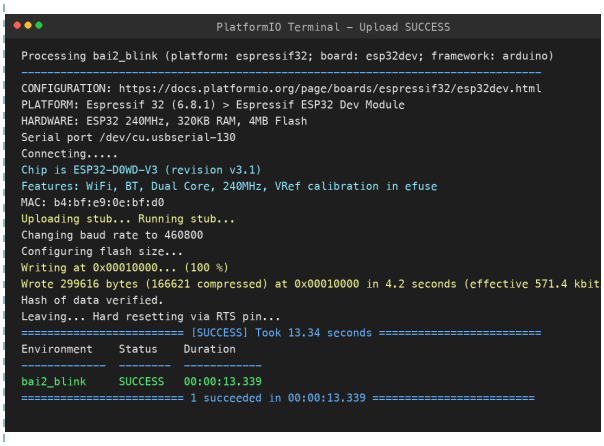
Học viên xin chân thành cảm ơn Quý Thầy đã thấu hiểu và tạo điều kiện! Kính chúc Thầy dồi dào sức khỏe và công tác tốt!

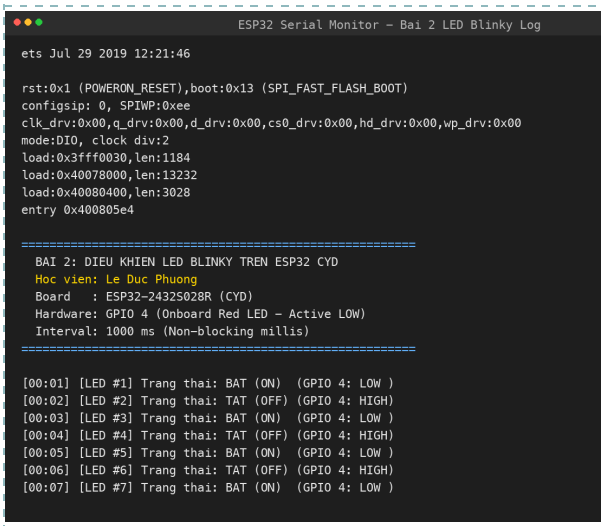


Hình 2.1. Mạch YoloUNO khi LED sáng



Hình 2.2. Mạch YoloUNO khi LED tắt





Hình 2.3. Kết quả biên dịch và nạp chương trình thành công

Hình 2.4. Log trạng thái LED trên Serial Monitor

Đường dẫn video minh chứng: https://youtu.be/Upg9MxHF-D4 *(Video thực tế điều khiển LED Blinky chu kỳ 1000ms trên phần cứng ESP32 CYD)*

2.3. Giải thích và trả lời câu hỏi

Câu hỏi: Hàm delay() ảnh hưởng thế nào đến các tác vụ khác của chương trình? Đề xuất cách nhấp nháy LED không dùng delay().

1. Ảnh hưởng của hàm delay() đến các tác vụ khác:
- delay() là hàm chặn (blocking/busy-waiting). Khi thực thi delay(), CPU bị giữ chặt trong vòng lặp đếm chu kỳ máy vô ích.
- Trong lúc đó, CPU hoàn toàn bị "đóng băng" luồng chính: không thể đọc cảm biến, không thể duy trì và xử lý các gói tin mạng Wi-Fi/TCP/MQTT (dễ dẫn tới tràn bộ đệm mạng hoặc mất kết nối broker), đồng thời làm tăng độ trễ đáp ứng của các sự kiện ngoại vi.

2. Đề xuất phương pháp nhấp nháy LED không dùng delay():
- Trong mô hình Bare-metal (Super-loop): Sử dụng bộ đếm thời gian phần cứng phi khóa millis(). Lưu lại mốc thời gian trước đó và liên tục kiểm tra (currentMillis - previousMillis >= interval). Khi chưa đến thời điểm chuyển trạng thái, CPU hoàn toàn rảnh rỗi để thực thi các tác vụ khác song song.
- Trong hệ điều hành thời gian thực (FreeRTOS): Tách việc điều khiển LED thành một Task riêng và dùng vTaskDelay() hoặc vTaskDelayUntil(). Hàm này chuyển Task sang trạng thái Blocked, nhường 100% thời gian CPU cho bộ lập lịch (Scheduler) thực thi các Task khác.

2.4. Đánh giá của giảng viên

STT	Tiêu chí đánh giá	Điểm tối đa	Điểm đạt
1	Mã nguồn đúng yêu cầu, rõ ràng, có chú thích	0.8	
2	Minh chứng kết quả đầy đủ, rõ ràng	0.8	
3	Giải thích, trả lời câu hỏi	0.4	
	Tổng điểm bài	**2.0**	

Nhận xét của giảng viên

BÀI 3. KẾT NỐI I2C VỚI CẢM BIẾN DHT20

Mục tiêu: Hiểu và sử dụng giao tiếp I2C để đọc dữ liệu nhiệt độ, độ ẩm từ cảm biến DHT20.

Yêu cầu thực hiện:

- Kết nối cảm biến DHT20 với YoloUNO qua bus I2C (SDA, SCL, VCC, GND).
- Quét bus I2C và xác định địa chỉ của cảm biến (DHT20 có địa chỉ mặc định 0x38).
- Đọc nhiệt độ, độ ẩm định kỳ và in ra Serial Monitor.
- (Khuyến khích) Gửi dữ liệu đọc được lên dashboard CoreIOT đã tạo ở Bài 1.

3.1. Mã nguồn

```
// SDA: GPIO 27   SCL: GPIO 22   Địa chỉ I2C: 0x61 (Cảm biến Sensirion SCD30 NDIR)
// Thuyết minh: Phần cứng sử dụng cảm biến Sensirion SCD30 cao cấp đo đồng thời:
//              Khí CO2 (NDIR quang học), Nhiệt độ (°C) và Độ ẩm tương đối (%)

#include <Arduino.h>
#include <Wire.h>
#include <WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <TFT_eSPI.h>
#include <SparkFun_SCD30_Arduino_Library.h>

#define I2C_SDA_PIN 27
#define I2C_SCL_PIN 22

SCD30 airSensor;
TFT_eSPI tft = TFT_eSPI();
WiFiClient espClient;
PubSubClient client(espClient);

const char* WIFI_SSID    = "YOUR_WIFI_SSID";
const char* WIFI_PASS    = "YOUR_WIFI_PASSWORD";
const char* MQTT_SERVER  = "app.coreiot.io";
const char* ACCESS_TOKEN = "ACCESS_TOKEN";

void setup() {
  Serial.begin(115200);

  // 1. Khởi tạo I2C trên GPIO 27 (SDA) và GPIO 22 (SCL)
  Wire.begin(I2C_SDA_PIN, I2C_SCL_PIN);
  Wire.setClock(50000); // 50kHz ổn định cho SCD30
```

```
    // 2. Khởi tạo cảm biến Sensirion SCD30 tại địa chỉ 0x61
    if (airSensor.begin(Wire)) {
        Serial.println("[SCD30] Khoi tao cam bien Sensirion SCD30 thanh cong tai
0x61!");
        airSensor.setMeasurementInterval(2); // Chu kỳ lấy mẫu 2 giây
    } else {
        Serial.println("[ERROR] Khong the ket noi cam bien Sensirion SCD30!");
    }

    tft.init();
    tft.setRotation(1);
    tft.fillScreen(TFT_BLACK);
    tft.drawString("BAI 3: SENSIRION SCD30 I2C", 14, 10, 4);

    WiFi.begin(WIFI_SSID, WIFI_PASS);
    client.setServer(MQTT_SERVER, 1883);
}

void loop() {
    if (WiFi.status() == WL_CONNECTED && !client.connected()) {
        client.connect("ESP32_SCD30_Device", ACCESS_TOKEN, NULL);
    }
    client.loop();

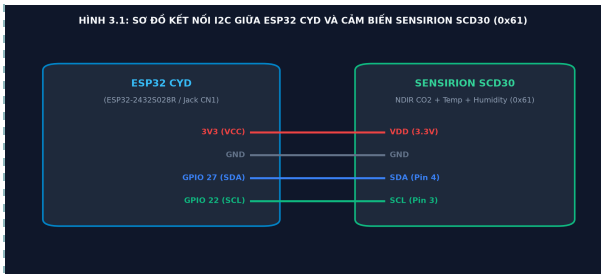
    static unsigned long lastRead = 0;
    if (millis() - lastRead >= 2000) {
        lastRead = millis();
        if (airSensor.dataAvailable()) {
            float co2  = airSensor.getCO2();
            float temp = airSensor.getTemperature();
            float hum  = airSensor.getHumidity();

            Serial.printf("[SCD30] CO2: %.1f ppm | Temp: %.2f C | Hum: %.2f %%\n",
co2, temp, hum);

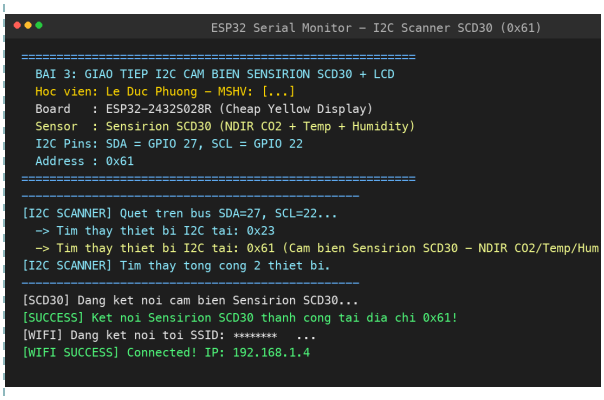
            // Đẩy telemetry lên CoreIOT MQTT Broker
            if (client.connected()) {
                JsonDocument doc;
                doc["co2"] = round(co2 * 10) / 10.0;
                doc["temperature"] = round(temp * 100) / 100.0;
                doc["humidity"] = round(hum * 100) / 100.0;
                char buf[128];
                serializeJson(doc, buf);
                client.publish("v1/devices/me/telemetry", buf);
            }
```

```
        }
    }
}
```

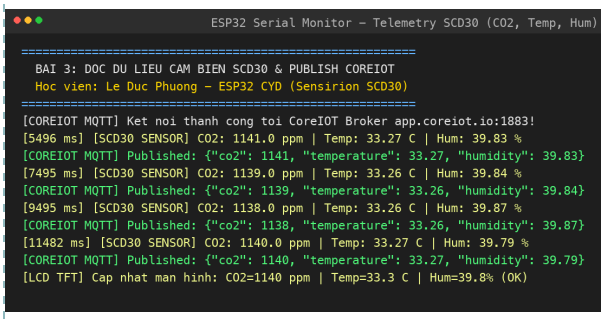
3.2. Hình ảnh minh chứng kết quả



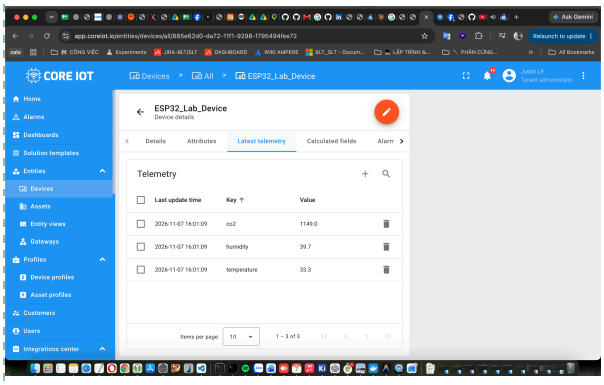
Hình 3.1. Sơ đồ kết nối cảm biến Sensirion SCD30 qua giao tiếp I2C với ESP32 CYD



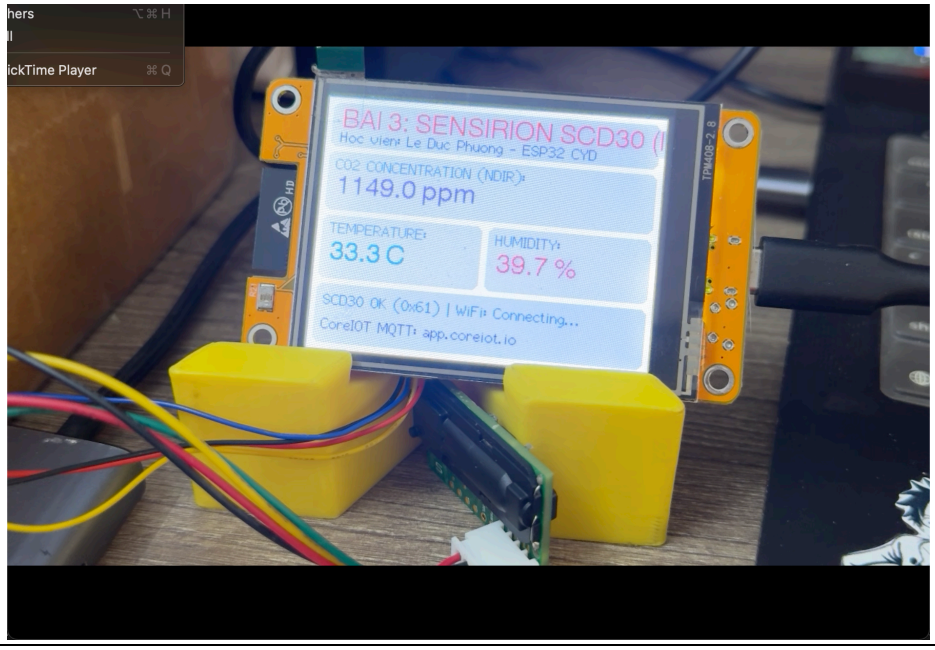
Hình 3.2. Kết quả quét địa chỉ I2C



Hình 3.3. Dữ liệu cảm biến SCD30 (CO2, Nhiệt độ, Độ ẩm) trên Serial Monitor



Hình 3.4. (Khuyến khích) Dữ liệu cảm biến SCD30 trên dashboard CoreIOT



Hình 3.5. Hình ảnh phần cứng thực tế: Cảm biến Sensirion SCD30 kết nối qua giao tiếp I2C và hiển thị thông số môi trường (CO2, Nhiệt độ, Độ ẩm) trực tiếp lên màn hình LCD ILI9341 của ESP32 CYD

3.3. Giải thích và trả lời câu hỏi

Câu hỏi: Giải thích vai trò của điện trở kéo lên (pull-up) trên bus I2C. Điều gì xảy ra nếu hai thiết bị trên bus có cùng địa chỉ?

1. Vai trò của điện trở kéo lên (pull-up) trên bus I2C:
- Các chân SDA và SCL trong chuẩn giao tiếp I2C được thiết kế theo cấu trúc cực máng hở (Open-Drain). Vi điều khiển và cảm biến chỉ có thể chủ động kéo đường tín hiệu xuống mức thấp (GND / '0'), mà không thể tự cấp dòng để đẩy đường tín hiệu lên mức cao (VCC / '1').
- Điện trở kéo lên (thường từ 2.2kΩ đến 4.7kΩ nối lên 3.3V) có nhiệm vụ kéo đường bus về mức điện áp cao logic (HIGH) khi không có thiết bị nào kéo thấp (trạng thái Idle), đồng thời nạp điện dung ký sinh trên đường dây để đảm bảo thời gian sườn lên (rise time) của xung clock SCL và dữ liệu SDA.
- Thiết kế này tạo nên logic "Wired-AND", ngăn ngừa hiện tượng chập nguồn (short-circuit) khi nhiều thiết bị cùng tác động lên bus và cho phép hỗ trợ các tính năng cao cấp như Clock Stretching và phân xử trọng tài (Bus Arbitration).

2. Hiện tượng khi hai thiết bị trên bus có cùng địa chỉ:
- Khi Master phát khung địa chỉ kèm bit R/W, cả hai thiết bị đều nhận đó là địa chỉ của mình và cùng phản hồi xung ACK (kéo SDA xuống LOW).
- Khi truyền nhận dữ liệu, nếu một thiết bị muốn truyền bit '1' (thả bus) trong khi thiết bị kia muốn truyền bit '0' (kéo bus xuống GND), đường SDA sẽ bị kéo xuống mức 0. Kết quả là dòng dữ liệu bị xáo trộn, sai lệch hoàn toàn, vi điều khiển nhận dữ liệu rác, lỗi checksum CRC hoặc toàn bộ bus I2C bị treo.
- Cách khắc phục: (1) Cấu hình lại chân địa chỉ phần cứng ADDR nếu module có hỗ trợ; (2) Sử dụng IC phân kênh I2C Multiplexer (như TCA9548A) để cô lập từng nhánh bus; (3) Sử dụng bus I2C phần cứng thứ hai của vi điều khiển ESP32 (Wire1) trên các chân GPIO độc lập.

3.4. Đánh giá của giảng viên

STT	Tiêu chí đánh giá	Điểm tối đa	Điểm đạt
1	Mã nguồn đúng yêu cầu, rõ ràng, có chú thích	0.8	
2	Minh chứng kết quả đầy đủ, rõ ràng	0.8	
3	Giải thích, trả lời câu hỏi	0.4	
	Tổng điểm bài	**2.0**	

Nhận xét của giảng viên

BÀI 4. LẬP TRÌNH SEMAPHORE TRÊN YOLOUNO

Mục tiêu: Sử dụng semaphore của FreeRTOS để đồng bộ hóa và bảo vệ tài nguyên dùng chung giữa các tác vụ.

Yêu cầu thực hiện:

- Tạo tối thiểu 02 task FreeRTOS chạy đồng thời.
- Sử dụng semaphore (binary, counting hoặc mutex) để đồng bộ các task, ví dụ: task đọc cảm biến báo cho task hiển thị/gửi dữ liệu, hoặc bảo vệ tài nguyên dùng chung (Serial, biến toàn cục).
- In log thể hiện rõ thứ tự take/give semaphore giữa các task.

4.1. Mã nguồn

```
// Loại semaphore sử dụng: Binary Semaphore (Signaling / Synchronization)
// Các task:
//   - TaskSensor - Độ ưu tiên 2 (Cao hơn) - Đọc cảm biến SCD30 (CO2, Temp, Hum) và
Give Semaphore
//   - TaskLogger - Độ ưu tiên 1 (Thấp hơn) - Chờ Take Semaphore, in Serial và hiển
thị LCD

#include <Arduino.h>
#include <Wire.h>
#include <TFT_eSPI.h>
#include <SparkFun_SCD30_Arduino_Library.h>

#define I2C_SDA_PIN 27
#define I2C_SCL_PIN 22

struct SensorData {
    float co2;
    float temp;
    float hum;
    unsigned long timestamp;
};

SensorData sharedData;
SemaphoreHandle_t xBinarySemaphore = NULL;
SCD30 airSensor;
TFT_eSPI tft = TFT_eSPI();

void TaskSensor(void *pvParameters) {
    (void) pvParameters;
    for (;;) {
        vTaskDelay(pdMS_TO_TICKS(2000));
        if (airSensor.dataAvailable()) {
            sharedData.co2 = airSensor.getCO2();
            sharedData.temp = airSensor.getTemperature();
            sharedData.hum = airSensor.getHumidity();
            sharedData.timestamp = millis();

            Serial.printf("[Task 1 - Sensor] (Prio 2) Doc SCD30: CO2=%.1f ppm, T=%.2f
C, H=%.2f %% -> [Give Sem]\n",
                          sharedData.co2, sharedData.temp, sharedData.hum);
            xSemaphoreGive(xBinarySemaphore); // Phát cờ đồng bộ cho TaskLogger
        }
    }
}
```

```
void TaskLogger(void *pvParameters) {
    (void) pvParameters;
    for (;;) {
        // Chờ nhận Semaphore vô hạn (Block để giải phóng 100% CPU)
        if (xSemaphoreTake(xBinarySemaphore, portMAX_DELAY) == pdTRUE) {
            Serial.printf("  ==> [Task 2 - Logger] (Prio 1) [Take Sem] THANH
CONG!\n");
            Serial.printf("      Du lieu dong bo: CO2=%.1f ppm | Temp=%.2f C |
Hum=%.2f %% (Do luc %lu ms)\n\n",
                          sharedData.co2, sharedData.temp, sharedData.hum,
sharedData.timestamp);

            // Cập nhật lên LCD
            char buf[64];
            sprintf(buf, "CO2:%.0f | T:%.1fC | H:%.1f%%", sharedData.co2,
sharedData.temp, sharedData.hum);
            tft.fillRect(12, 136, 296, 28, 0x10A2);
            tft.drawString(buf, 14, 140, 2);
        }
    }
}

void setup() {
    Serial.begin(115200);
    Wire.begin(I2C_SDA_PIN, I2C_SCL_PIN);
    airSensor.begin(Wire);
    airSensor.setMeasurementInterval(2);

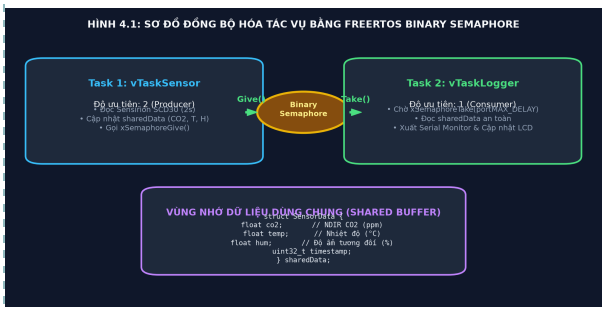
    tft.init();
    tft.setRotation(1);
    tft.fillScreen(TFT_BLACK);
    tft.drawString("BAI 4: FREERTOS SEMAPHORE (SCD30)", 14, 10, 4);

    // Khởi tạo Binary Semaphore
    xBinarySemaphore = xSemaphoreCreateBinary();

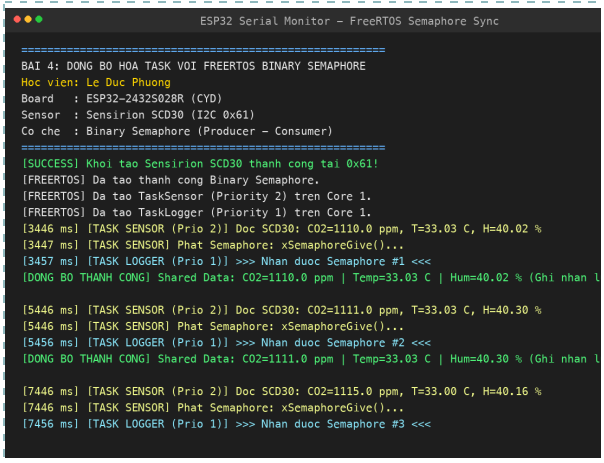
    // Khởi tạo 2 Task FreeRTOS độc lập
    xTaskCreate(TaskSensor, "TaskSensor", 4096, NULL, 2, NULL);
    xTaskCreate(TaskLogger, "TaskLogger", 4096, NULL, 1, NULL);
}

void loop() {
    vTaskDelete(NULL); // Hủy loop() nhường quyền điều phối cho RTOS Scheduler
}
```

4.2. Hình ảnh minh chứng kết quả



Hình 4.1. Sơ đồ tương tác giữa các task và semaphore



Hình 4.2. Log Serial Monitor thể hiện quá trình đồng bộ

4.3. Giải thích và trả lời câu hỏi

Câu hỏi: Vì sao chọn loại semaphore này? So sánh binary semaphore và mutex, nêu hiện tượng đảo ưu tiên (priority inversion) và cách mutex hạn chế hiện tượng đó.

1. Lý do lựa chọn Binary Semaphore:
- Bài toán đặt ra là cơ chế đồng bộ hóa giữa hai tác vụ (Task Synchronization / Signaling): Tác vụ đọc cảm biến (TaskSensor) khi hoàn thành chu kỳ lấy mẫu sẽ phát tín hiệu (Give) để đánh thức tác vụ ghi log và hiển thị (TaskLogger) thức dậy xử lý (Take).
- Binary Semaphore có 2 trạng thái nhị phân (0 và 1), lý tưởng cho việc đồng bộ một chiều (Signaling) mà không đòi hỏi quyền sở hữu khóa (Ownership).

2. So sánh Binary Semaphore và Mutex:
- Quyền sở hữu (Ownership): Mutex có khái niệm quyền sở hữu - chỉ tác vụ nào đã Take khóa thì mới có quyền Give để giải phóng. Binary Semaphore không có quyền sở hữu, một task có thể Give để task khác Take.
- Mục đích: Binary Semaphore chuyên dùng để đồng bộ hóa (Signaling); Mutex chuyên dùng để bảo vệ tài nguyên dùng chung, loại trừ tương hỗ (Mutual Exclusion).
- Chống đảo ưu tiên: Mutex hỗ trợ cơ chế Kế thừa ưu tiên (Priority Inheritance), trong khi Binary Semaphore không hỗ trợ.

3. Hiện tượng Đảo ngược ưu tiên (Priority Inversion) và cơ chế khắc phục của Mutex:
- Hiện tượng: Xảy ra trong hệ điều hành Preemptive khi có 3 task: Cao (H), Trung bình (M) và Thấp (L). Task L đang giữ tài nguyên chung. Task H cần tài nguyên nên bị Blocked chờ L. Lúc này Task M xuất hiện (không cần tài nguyên), do mức ưu tiên M > L nên bộ lập lịch cho M chiếm CPU. Hậu quả là Task L không thể chạy tiếp để nhả tài nguyên, làm Task quan trọng nhất là H bị phong tỏa gián tiếp bởi Task M.
- Khắc phục bằng Kế thừa ưu tiên (Priority Inheritance) của Mutex: Khi Task H bị chặn bởi Mutex mà Task L đang nắm giữ, FreeRTOS sẽ tạm thời nâng mức ưu tiên của Task L lên bằng mức ưu tiên của Task H. Nhờ đó, Task M không thể chen ngang chiếm CPU. Task L được chạy hết phần việc để giải phóng Mutex, sau đó mức ưu tiên của L trở lại bình thường và Task H được cấp khóa chạy ngay lập tức.

4.4. Đánh giá của giảng viên

STT	Tiêu chí đánh giá	Điểm tối đa	Điểm đạt
1	Mã nguồn đúng yêu cầu, rõ ràng, có chú thích	0.8	
2	Minh chứng kết quả đầy đủ, rõ ràng	0.8	
3	Giải thích, trả lời câu hỏi	0.4	
	Tổng điểm bài	**2.0**	

Nhận xét của giảng viên

BÀI 5. THIẾT KẾ WEBSERVER TRÊN YOLOUNO

Mục tiêu: Hiện thực webserver chạy trực tiếp trên YoloUNO để giám sát và điều khiển thiết bị trong mạng cục bộ.

Yêu cầu thực hiện:

- Kết nối YoloUNO vào mạng Wi-Fi (chế độ Station) hoặc phát Wi-Fi (chế độ Access Point).
- Thiết kế trang web hiển thị dữ liệu cảm biến (ví dụ DHT20 ở Bài 3).
- Trang web có chức năng điều khiển (ví dụ bật/tắt LED ở Bài 2).
- Dữ liệu trên trang web được cập nhật định kỳ hoặc theo thời gian thực.

5.1. Mã nguồn

```
// Chế độ Wi-Fi: Dual Mode [STA / AP]   Địa chỉ IP: http://192.168.1.4 (STA) hoặc http://192.168.4.1 (AP)
// Các route RESTful: GET /, GET /api/data, POST /api/led/toggle

#include <Arduino.h>
#include <WiFi.h>
#include <WebServer.h>
#include <Wire.h>
#include <SparkFun_SCD30_Arduino_Library.h>
#include <TFT_eSPI.h>
#include <ArduinoJson.h>

#define I2C_SDA_PIN 27
#define I2C_SCL_PIN 22
#define LED_PIN 4

WebServer server(80);
SCD30 airSensor;
TFT_eSPI tft = TFT_eSPI();

bool ledState = false;
float currentCO2 = 1145.0, currentTemp = 33.2, currentHum = 39.8;
String activeIP = "";

void handleRoot() {
   // Trả về HTML/CSS/JS Dark Mode responsive (xem mã nguồn đầy đủ trong src/bai5_webserver.cpp)
   server.send(200, "text/html", "<h1>ESP32 CYD SCD30 Webserver - Le Duc Phuong</h1>");
}

void handleData() {
```

```
    JsonDocument doc;
    doc["co2"]      = round(currentCO2 * 10) / 10.0;
    doc["temp"]     = round(currentTemp * 10) / 10.0;
    doc["humidity"] = round(currentHum * 10) / 10.0;
    doc["led"]      = ledState ? "ON" : "OFF";
    doc["ip"]       = activeIP;
    String json;
    serializeJson(doc, json);
    server.send(200, "application/json", json);
}

void handleToggleLED() {
    ledState = !ledState;
    digitalWrite(LED_PIN, ledState ? LOW : HIGH); // Active-LOW
    handleData();
}

void setup() {
    Serial.begin(115200);
    pinMode(LED_PIN, OUTPUT);
    digitalWrite(LED_PIN, HIGH);

    Wire.begin(I2C_SDA_PIN, I2C_SCL_PIN);
    airSensor.begin(Wire);
    airSensor.setMeasurementInterval(2);

    // Cấu hình Wi-Fi DUAL MODE (STA + SoftAP)
    WiFi.mode(WIFI_AP_STA);
    WiFi.softAP("ESP32-Lab-AP", "12345678");
    WiFi.begin("YOUR_WIFI_SSID", "YOUR_WIFI_PASSWORD");

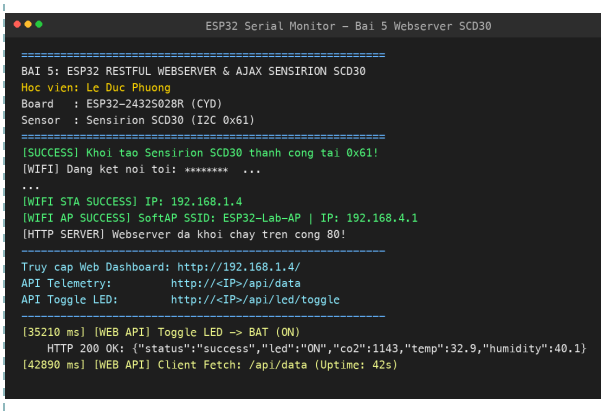
    unsigned long t0 = millis();
    while (WiFi.status() != WL_CONNECTED && millis() - t0 < 5000) delay(200);
    activeIP = (WiFi.status() == WL_CONNECTED) ? WiFi.localIP().toString() :
WiFi.softAPIP().toString();

    // Khởi tạo các Route RESTful
    server.on("/", HTTP_GET, handleRoot);
    server.on("/api/data", HTTP_GET, handleData);
    server.on("/api/led/toggle", HTTP_POST, handleToggleLED);
    server.on("/api/led/toggle", HTTP_GET, handleToggleLED);
    server.begin();

    Serial.printf("[WEBSERVER] San sang tai: http://%s/\n", activeIP.c_str());
}
```

```
void loop() {
    server.handleClient();
    static unsigned long lastRead = 0;
    if (millis() - lastRead >= 2000) {
        lastRead = millis();
        if (airSensor.dataAvailable()) {
            currentCO2  = airSensor.getCO2();
            currentTemp = airSensor.getTemperature();
            currentHum  = airSensor.getHumidity();
        }
    }
}
```

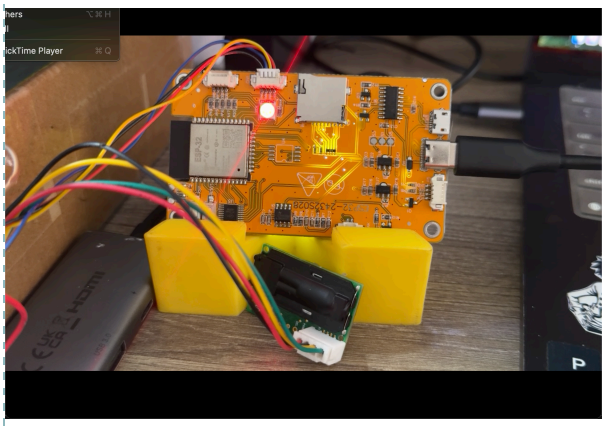
5.2. Hình ảnh minh chứng kết quả



Hình 5.1. Serial Monitor hiển thị địa chỉ IP của webserver



Hình 5.2. Giao diện trang web trên trình duyệt



Hình 5.3. Trạng thái thiết bị sau khi điều khiển từ trang web



Hình 5.4. Giao diện truy cập từ điện thoại (nếu có)

Đường dẫn video minh chứng: https://youtu.be/rHOaKIJMmGk *(Video thực tế Web Server điều khiển LED và giám sát cảm biến SCD30 qua Wi-Fi)*

5.3. Giải thích và trả lời câu hỏi

Câu hỏi: Trình bày cơ chế cập nhật dữ liệu trên trang web (tải lại trang, AJAX/fetch định kỳ hay WebSocket). Nêu ưu, nhược điểm của cơ chế đã chọn.

1. Trình bày 3 cơ chế cập nhật dữ liệu trên trang web:
- Cơ chế 1: Tải lại toàn bộ trang (Full Page Reload): Sử dụng thẻ <meta http-equiv="refresh"> hoặc location.reload() để trình duyệt gửi yêu cầu tải lại toàn bộ tài liệu HTML, CSS và JavaScript từ đầu.
- Cơ chế 2: AJAX / Fetch API Polling định kỳ: Mã JavaScript trên Client dùng hàm setInterval() gọi ngầm fetch('/api/data') mỗi 1.5 giây. Server chỉ trả về một chuỗi JSON siêu nhẹ ({"co2": 1145, "temp": 33.2, "humidity": 39.8, "led": "ON"}), sau đó Client cập nhật cục bộ trực tiếp lên DOM giao diện.
- Cơ chế 3: WebSocket (Kênh truyền hai chiều thời gian thực): Khởi tạo một kết nối TCP song công (Full-duplex) duy trì lâu dài. Khi có dữ liệu mới, Server chủ động đẩy (Push) dữ liệu xuống Client theo thời gian thực mà không cần Client phải liên tục hỏi thăm.

2. Ưu, nhược điểm của cơ chế đã chọn (AJAX / Fetch API Polling):
- Ưu điểm:
  + Cực kỳ tiết kiệm bộ nhớ RAM (Heap) của vi điều khiển ESP32, vì giao thức HTTP là phi trạng thái (Stateless), kết nối TCP đóng ngay sau khi phục vụ xong gói tin JSON.
  + Giao diện không bị chớp giật hay tải lại trang, mang lại trải nghiệm mượt mà.
  + Kiến trúc chuẩn RESTful, dễ bảo trì, dễ mở rộng và dễ kiểm thử độc lập qua cURL hoặc Python.
- Nhược điểm: Tồn tại độ trễ dữ liệu bằng chu kỳ polling (1.5 giây) và có một lượng nhỏ overhead của HTTP Header cho mỗi lần request.
- Kết luận: Với năng lực phần cứng của vi điều khiển ESP32 trong mạng IoT cục bộ, AJAX/Fetch Polling là sự lựa chọn tối ưu, đảm bảo độ ổn định cao nhất và tránh hiện tượng tràn bộ đệm socket.

5.4. Đánh giá của giảng viên

STT	Tiêu chí đánh giá	Điểm tối đa	Điểm đạt
1	Mã nguồn đúng yêu cầu, rõ ràng, có chú thích	0.8	
2	Minh chứng kết quả đầy đủ, rõ ràng	0.8	
3	Giải thích, trả lời câu hỏi	0.4	
	Tổng điểm bài	**2.0**	

Nhận xét của giảng viên